

Blockchain Interoperability Overview

This note compiles key concepts from five lecture-based PDFs, providing a unified look at blockchain interoperability, consensus designs, Hyperledger Fabric chaincode, Byzantine agreement protocols, and permissioned-model consensus.

Content Sources

- Section 1: Overview of **interoperability**, **cross-chain protocols**, and **state-relay mechanisms** from a 76-page PDF presentation. ([source](#))
- Section 2: Detailed look at **ByzCoin**, **PoW-PBFT hybrid**, and **scalable consensus** from a 97-page PDF presentation. ([source](#))
- Section 3: Explanation of **Hyperledger Fabric**, **chaincode development**, and **transaction flow** from an 80-page PDF presentation. ([source](#))
- Section 4: Insight into **Byzantine Agreement**, **protocol design**, and **practical examples** from a 96-page PDF presentation. ([source](#))
- Section 5: Discussion of **permissioned blockchains**, **consensus mechanisms**, and **membership service provider** from a 177-page PDF presentation. ([source](#))

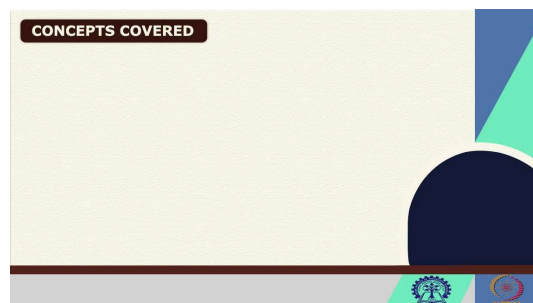
Section 1: Blockchain Interoperability – A Complete Overview

This section covers blockchain interoperability mechanisms, cross-chain asset transfers, atomic swaps, and verifiable credentials from a 76-page presentation ([source](#))



Course Context

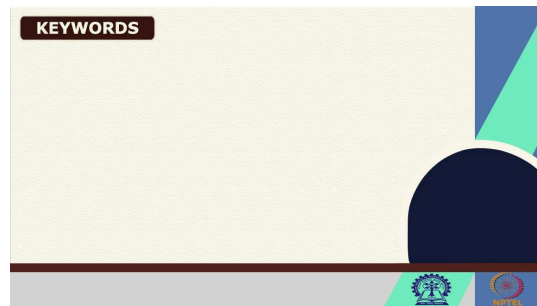
- **Course:** *Blockchain and its Applications*
- **Lecturer:** Prof. Shamik Sural, Department of Computer Science & Engineering, IIT Kharagpur
- **Lecture Series:** Lectures 46–50 covering blockchain interoperability and Hyperledger Indy



Core Concepts Covered

Topic	Sub-topics
-------	------------

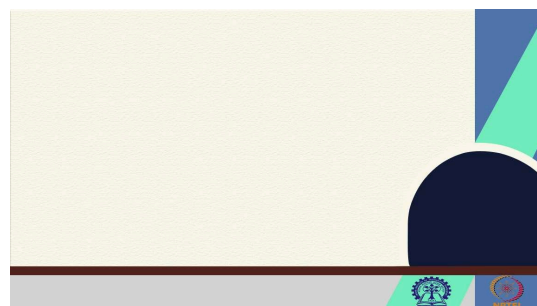
Basic Concept of Asset & Data Transfer	Asset transfer in permissionless blockchains, Cross-chain asset transfer, Trusted third party (TTP)
Interoperability	Permissionless vs. permissioned blockchains, Public-blockchain silos
Asset Exchange	Atomic swap, HTLC, Hash-lock & time-lock mechanisms
Advanced Interoperability	Multi-party cross-chain swap, Permissioned blockchain data transfer, Hyperledger Fabric inter-networking
Identity & Credential Management	Hyperledger Indy, DIDs, Verifiable Credentials & Presentations



Key Terminology

Term	Definition
Interoperability	Ability of distinct blockchain networks to exchange assets, data, or state
Asset Transfer	Moving ownership of a token/coin from one address to another, possibly across chains
Cross-Chain Transfer	Transfer that involves two (or more) independent blockchains
Trusted Third Party (TTP)	Centralised entity that temporarily holds assets to facilitate cross-chain moves
Asset Exchange	Simultaneous swap of assets on two different chains (e.g., BTC ↔ ETH)
Atomic Swap	A trust-less exchange where either both legs execute or none do
Hash-Lock / Time-Lock	Cryptographic primitives that enforce conditional spending
Hashed Timelock Contract (HTLC)	A contract combining a hash-lock and a time-lock to achieve atomicity

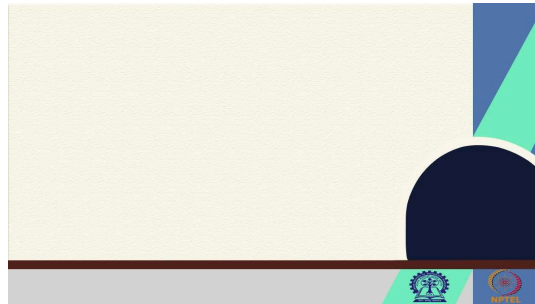
Interoperability in Permissionless vs. Permissioned Blockchains



Permissionless (Public) Blockchains	Permissioned Blockchains
• Open to anyone to join and validate	• Participation limited to approved entities

• Asset transfer driven by native cryptocurrencies (e.g., BTC, ETH)	• Data transfer driven by consensus protocols (e.g., Fabric, Indy)
• No central authority; security via PoW/PoS	• Governance by known members; often faster finality
• Example: Bitcoin, Ethereum	• Example: Hyperledger Fabric, Hyperledger Indy

Public Blockchains as Isolated Silos



Key insight: Over 2,000 cryptocurrencies exist, each on its own network with different protocols, standards, and consensus mechanisms. These networks **do not natively communicate**—they operate as "silos."

🔗 Cross-Chain Asset Transfer – The Naïve Picture

Scenario	Description
Alice → Bob (BTC → ETH)	Alice holds 1 BTC on Bitcoin and wants to give Bob 33 ETH on Ethereum
Direct Transfer?	Not possible —the two ledgers are independent
Third-Party Involvement	Alice must rely on a Trusted Third Party (TTP) that controls ≥33 ETH on Ethereum to fulfill Bob's receipt

🏦 Trusted Third Party (TTP) Based Asset Transfer

How it works:

1. Alice deposits her BTC with a centralized exchange (e.g., Binance, Coinbase)
2. The exchange holds the BTC and, after confirming receipt, **credits Bob** with corresponding ETH on its internal ledger
3. The exchange then **withdraws** the ETH to Bob's external Ethereum address

Advantages:

- Very fast once deposits are confirmed (often milliseconds)

Security & Trust Issues:

Incident	Loss	Year
Mt. Gox exchange hack	650,000 BTC lost	2014
Bitfinex hack	≈120,000 BTC stolen	2016

Critical vulnerability: Centralized exchanges become **single points of failure**. Users **surrender custody** of private keys to the exchange.

References:

- Bitcointalk thread on exchange hacks: <https://bitcointalk.org/index.php?topic=576337>

🔗 Asset Exchange & the Need for Atomicity

Problem Statement:

- When two parties exchange assets on different chains **without an escrow**, either party can abort after receiving the counter-asset, leading to loss
- Additional challenges: synchronization between chains, fluctuating exchange rates, race conditions

Goal: Ensure the swap is **atomic**—either **both transfers happen** or **neither** does.

Atomic Swaps – The Core Idea

Definition of Atomicity

"An atomic transaction is an indivisible series of operations such that either all occur, or none occurs."

Atomic Swap Protocol Guarantees:

- All-conforming parties** complete the swap
- Deviating parties** cannot end up better off; honest parties are protected
- No coalition** can profit by deviating (game-theoretic stability)

High-Level Procedure (PODC '18)

- Smart contracts** are instantiated on **both** blockchains, each locking the respective asset
- Both contracts share a **common spending condition** (the hash-lock)
- When the condition is satisfied, **both contracts release** funds **simultaneously**

Illustrative Diagram (textual representation):

```
Bitcoin Network          Ethereum Network
-----
Alice creates Contract A
- Locks 1 BTC
- Hashlock = H
- Timelock = 4Δ

Bob creates Contract B
- Locks 33 ETH
- Same Hashlock = H
- Timelock = 2Δ (shorter)

Bob redeems Contract A by revealing pre-image (secret)
→ Secret is broadcast → Alice can now redeem Contract B
```

Hash-Lock & Time-Lock Primitives

Primitive	Purpose	Mechanism
Hash-Lock	Prevents spending until a secret value is disclosed	The contract stores $H = \text{hash}(\text{secret})$. When anyone presents the pre-image (secret) that hashes to H , the funds become spendable
Time-Lock	Prevents spending until a specific block height or timestamp	The contract includes a timeout (Δ). If the timeout expires before the secret is revealed, the locked funds are refunded to the original owner

Example – Hash-Lock:

- Alice picks secret **"I love strawberries"**
- Computes hash $H = \text{f1b81571baac90bed544d1910f79ea5c31fa4509}$
- Creates a Bitcoin contract: *"Pay Bob 1 BTC if secret is revealed; else refund after timeout"*

Example – Time-Lock:

- Alice creates a contract with timeout 2Δ
- After 2Δ (e.g., 2 hours or 2 blocks), if the secret never appears, the contract automatically returns the BTC to Alice

Section 2: ByzCoin and Algorand – Scalable Consensus Protocols

This section covers ByzCoin's hybrid PoW-PBFT approach and Algorand's pure proof-of-stake consensus with cryptographic sortition from a 97-page presentation ([source](#))



Lecture 41 – ByzCoin

Overview

Goal: Combine **Proof-of-Work (PoW)** mining with **Practical Byzantine Fault Tolerance (PBFT)** to obtain a blockchain that is both **scalable** and **strongly consistent**.

Key Questions:

1. How far can scalability be pushed?
2. Can we open the PBFT consensus group while keeping low communication overhead?

Requirements for a Blockchain Consensus Protocol

Requirement	Description
Byzantine fault tolerant	System must remain correct despite malicious participants across multiple administrative domains
Strong consistency	All honest replicas must agree on the same block order (no forks)
Scalable transaction throughput	Ability to process more transactions per unit time as demand grows
Scalable network size	Ability to increase the number of nodes without degrading performance

Limitations of Existing Protocols

Bitcoin-NG (Key-Block Fault)

- **Faulty key block** is only verified at the *end* of a round, allowing a malicious miner to insert many correct micro-blocks after a faulty micro-block
- This creates **overhead** and can cause forks

PBFT Issues

Problem	Impact
Static consensus group	Nodes must be known in advance because PBFT relies on all-to-all message passing
Scalability bottleneck: $O(n^2)$ communication	High message overhead

Scalability bottleneck: $O(n)$ verification	Each node must verify all signatures
Key management	PBFT uses MACs, requiring shared secret keys among all participants – vulnerable to key leakage
Sybil attacks	An adversary can create many pseudonymous identities to meet the $3f+1$ requirement, undermining security

Opening the Consensus Group

Mechanism	How it works
PoW-based membership proof	Miners earn a <i>share</i> each time they find a new block; the share proves their right to join the BFT trustees
Sliding window	A fixed-size window of recent shares defines the active consensus group, allowing dynamic membership
Balance of power	The window ensures no single miner can dominate the BFT group for long

Merging PoW with BFT – Core Ideas

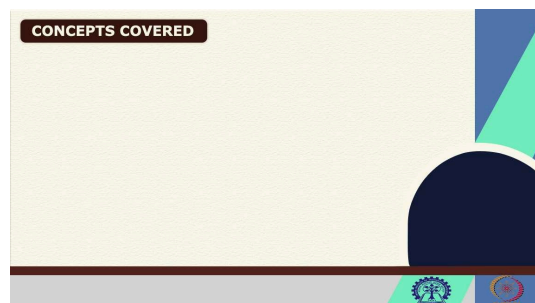
1. **Witness Consigners (CoSi)** – Each micro-block is validated by a *set of* witness cosigners selected from the current consensus group
2. **Collective Signing (CoSi)** – A scalable tree-based protocol that aggregates signatures, turning $O(n)$ verification into $O(1)$

Improving BFT Efficiency

Problem	Solution
$O(n)$ communication	Use tree-based multicast → information spreads in $O(\log n)$ hops
$O(n)$ verification	Deploy Schnorr multi-signatures → aggregated signature verified in constant time
Combined	CoSi (Communication trees + multi-signatures) yields low-latency, high-throughput consensus

ByzCoin Performance & Summary

Aspect	Achievement
Fault tolerance	Eliminates Bitcoin-NG's faulty-microblock problem by having PBFT-style verification of each micro-block
Scalability	Open consensus group via PoW shares + CoSi reduces both communication and verification costs, enabling near-Internet-scale operation
Core take-away	ByzCoin demonstrates that PoW can be used for membership while PBFT (enhanced by CoSi) provides strong consistency



Core Motivation

Challenge: Overcome the **Blockchain Performance Triangle** (security $\leftrightarrow$ decentralization $\leftrightarrow$ scalability) **Goal:** Provide **Internet-scale throughput** while preserving **BFT-level security** and **finality** (no forks)

Key Technical Contributions

Concept	Description
Cryptographic Sortition	Each user independently decides whether to participate in a given round, based on a Verifiable Random Function (VRF)
BA* (Byzantine Agreement)	A two-phase agreement protocol (Tentative $\rightarrow$ Final Consensus) that works under <i>weak synchrony</i> assumptions
Pure Proof-of-Stake (PPoS)	Consensus weight is proportional to the amount of money (stake) a user holds, not to computational work

Cryptographic Sortition Details

VRF Output: (hash, proof) $\leftarrow$ VRF_sk(x) where:

- x = input string (e.g., previous round seed)
- (pk, sk) = public/secret key pair
- **hash:** deterministic, unpredictable value
- **proof:** allows anyone to verify that the hash was correctly generated

Selection Process:

Parameter	Purpose
seed	Publicly known random value published at round r using VRFs from round r-1
threshold	Determines the expected number of selected users
role	Either <i>block proposer</i> or <i>committee member</i>
weights	w = individual stake; W = total stake of all users

A user may act as up to j "sub-users" if their stake is large enough, increasing their selection probability.

Algorand Consensus Flow

1. **Random user selection** – A user is selected (via sortition) to **propose a block**
2. **Block propagation** – The proposed block is gossiped across the network
3. **Committee selection** – A small random committee ($\approx 10k$ users) is selected to run **BA*** on the block
4. **Byzantine Agreement (BA*)** – Two-phase protocol:
 - **Tentative Consensus** – May produce multiple candidate blocks under weak synchrony
 - **Final Consensus** – Guarantees a single block when the network becomes strongly synchronous
5. **Signature aggregation** – Committee members sign the agreed block; signatures are aggregated for compact finality

Synchrony Assumptions

Synchrony Type	Characteristics
Strong Synchrony	$\geq 95\%$ of honest users receive messages within a known bound; guarantees liveness and enables final consensus in a single round
Weak Synchrony	The network may be fully asynchronous for arbitrary periods, but eventually experiences a <i>strongly synchronous</i> interval. Algorand remains safe (no double-spends) under weak synchrony and eventually reaches finality when the strong period occurs

BA* – Detailed Phases

Phase	Property
-------	----------

Final Consensus	If a user reaches final consensus, every honest user that reaches either final or tentative consensus in the same round must have the <i>same</i> block
Tentative Consensus	May result in different honest users agreeing on different correct blocks when the network is only weakly synchronous. The protocol recovers by launching the next BA* round; the rotating committee (thanks to sortition) prevents persistent forks

Conclusions on Algorand

Aspect	Achievement
Scalability	Supports millions of users with ≈ 1 minute latency
Security	BFT-style guarantees against a powerful adversary ($\leq 1/3$ of stake can be malicious)
Finality	Near-instant finality with negligible fork probability
Trade-off	Requires a <i>large</i> active user base to maintain security; the protocol's security hinges on honest-majority of stake

Section 2: ByzCoin and Algorand – Scalable Consensus Protocols

This section covers ByzCoin's hybrid PoW-PBFT approach and Algorand's pure proof-of-stake consensus with cryptographic sortition from a 97-page presentation ([source](#))



Lecture 41 – ByzCoin

Overview

Goal: Combine **Proof-of-Work (PoW)** mining with **Practical Byzantine Fault Tolerance (PBFT)** to obtain a blockchain that is both **scalable** and **strongly consistent**.

Key Questions:

1. How far can scalability be pushed?
2. Can we open the PBFT consensus group while keeping low communication overhead?

Requirements for a Blockchain Consensus Protocol

Requirement	Description
Byzantine fault tolerant	System must remain correct despite malicious participants across multiple administrative domains
Strong consistency	All honest replicas must agree on the same block order (no forks)
Scalable transaction throughput	Ability to process more transactions per unit time as demand grows

Scalable network size	Ability to increase the number of nodes without degrading performance
-----------------------	---

Limitations of Existing Protocols

Bitcoin-NG (Key-Block Fault)

- **Faulty key block** is only verified at the *end* of a round, allowing a malicious miner to insert many correct micro-blocks after a faulty micro-block
- This creates **overhead** and can cause forks

PBFT Issues

Problem	Impact
Static consensus group	Nodes must be known in advance because PBFT relies on all-to-all message passing
Scalability bottleneck: $O(n^2)$ communication	High message overhead
Scalability bottleneck: $O(n)$ verification	Each node must verify all signatures
Key management	PBFT uses MACs, requiring shared secret keys among all participants – vulnerable to key leakage
Sybil attacks	An adversary can create many pseudonymous identities to meet the $3f+1$ requirement, undermining security

Opening the Consensus Group

Mechanism	How it works
PoW-based membership proof	Miners earn a <i>share</i> each time they find a new block; the share proves their right to join the BFT trustees
Sliding window	A fixed-size window of recent shares defines the active consensus group, allowing dynamic membership
Balance of power	The window ensures no single miner can dominate the BFT group for long

Merging PoW with BFT – Core Ideas

1. **Witness Consigners (CoSi)** – Each micro-block is validated by a *set of* witness cosigners selected from the current consensus group
2. **Collective Signing (CoSi)** – A scalable tree-based protocol that aggregates signatures, turning $O(n)$ verification into $O(1)$

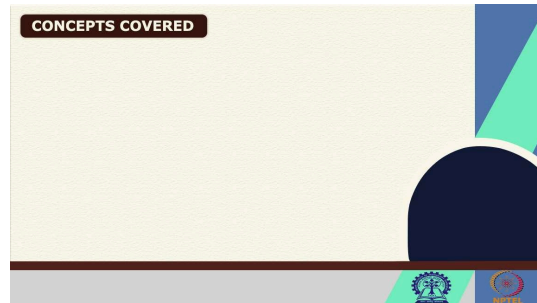
Improving BFT Efficiency

Problem	Solution
$O(n)$ communication	Use tree-based multicast → information spreads in $O(\log n)$ hops
$O(n)$ verification	Deploy Schnorr multi-signatures → aggregated signature verified in constant time
Combined	CoSi (Communication trees + multi-signatures) yields low-latency, high-throughput consensus

ByzCoin Performance & Summary

Aspect	Achievement
Fault tolerance	Eliminates Bitcoin-NG's faulty-microblock problem by having PBFT-style verification of each micro-block
Scalability	Open consensus group via PoW shares + CoSi reduces both communication and verification costs, enabling near-Internet-scale operation

Core take-away	ByzCoin demonstrates that PoW can be used for membership while PBFT (enhanced by CoSi) provides strong consistency
----------------	--



Lecture 42 – Algorand

Core Motivation

Challenge: Overcome the **Blockchain Performance Triangle** (security $\leftrightarrow$ decentralization $\leftrightarrow$ scalability) **Goal:** Provide **Internet-scale throughput** while preserving **BFT-level security** and **finality** (no forks)

Key Technical Contributions

Concept	Description
Cryptographic Sortition	Each user independently decides whether to participate in a given round, based on a Verifiable Random Function (VRF)
BA* (Byzantine Agreement)	A two-phase agreement protocol (Tentative $\rightarrow$ Final Consensus) that works under <i>weak synchrony</i> assumptions
Pure Proof-of-Stake (PPoS)	Consensus weight is proportional to the amount of money (stake) a user holds, not to computational work

Cryptographic Sortition Details

VRF Output: {hash, proof} $\leftarrow$ VRF_sk(x) where:

- x = input string (e.g., previous round seed)
- (pk, sk) = public/secret key pair
- **hash:** deterministic, unpredictable value
- **proof:** allows anyone to verify that the hash was correctly generated

Selection Process:

Parameter	Purpose
seed	Publicly known random value published at round r using VRFs from round r-1
threshold	Determines the expected number of selected users
role	Either <i>block proposer</i> or <i>committee member</i>
weights	w = individual stake; W = total stake of all users

A user may act as up to j "sub-users" if their stake is large enough, increasing their selection probability.

Algorand Consensus Flow

1. **Random user selection** – A user is selected (via sortition) to **propose a block**
2. **Block propagation** – The proposed block is gossiped across the network
3. **Committee selection** – A small random committee ($\approx 10k$ users) is selected to run **BA*** on the block
4. **Byzantine Agreement (BA*)** – Two-phase protocol:

- **Tentative Consensus** – May produce multiple candidate blocks under weak synchrony
 - **Final Consensus** – Guarantees a single block when the network becomes strongly synchronous
5. **Signature aggregation** – Committee members sign the agreed block; signatures are aggregated for compact finality

Synchrony Assumptions

Synchrony Type	Characteristics
Strong Synchrony	≥95% of honest users receive messages within a known bound; guarantees liveness and enables final consensus in a single round
Weak Synchrony	The network may be fully asynchronous for arbitrary periods, but eventually experiences a <i>strongly synchronous</i> interval. Algorand remains safe (no double-spends) under weak synchrony and eventually reaches finality when the strong period occurs

BA* – Detailed Phases

Phase	Property
Final Consensus	If a user reaches final consensus, every honest user that reaches either final or tentative consensus in the same round must have the <i>same</i> block
Tentative Consensus	May result in different honest users agreeing on different correct blocks when the network is only weakly synchronous. The protocol recovers by launching the next BA* round; the rotating committee (thanks to sortition) prevents persistent forks

Conclusions on Algorand

Aspect	Achievement
Scalability	Supports millions of users with ≈1 minute latency
Security	BFT-style guarantees against a powerful adversary (≤1/3 of stake can be malicious)
Finality	Near-instant finality with negligible fork probability
Trade-off	Requires a <i>large</i> active user base to maintain security; the protocol's security hinges on honest-majority of stake

Section 3: Hyperledger Fabric – Chaincode and DApp Development

This section covers Hyperledger Fabric's chaincode architecture, transaction flow, and practical DApp development using Node.js SDK from an 80-page presentation ([source](#))

Lecture 36 – Hyperledger Fabric 3

Instructor: Bishakh Chandra Ghosh (CSE, IIT Kharagpur)

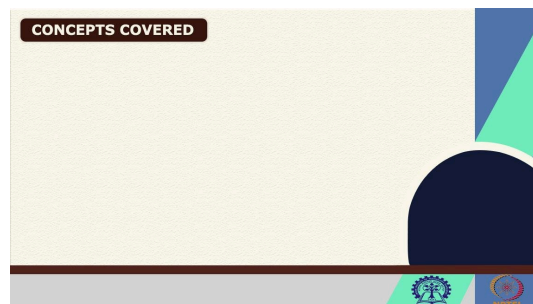
Introduction

- **Topic Overview:** Hyperledger Fabric chaincode, transaction flow, and how to write your own chaincode
- The lecture opens with the NPTEL-styled title slide that identifies the institute and the speaker



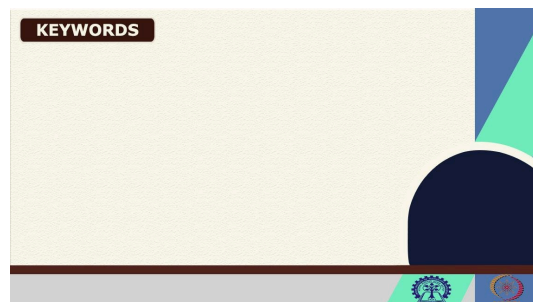
Core Concepts Covered

Concept	Brief Description
Fabric	Permissioned blockchain platform that separates transaction ordering from execution
Chaincode	Fabric's term for smart contracts – packaged business logic that runs on peers
Fabric Transactions	End-to-end flow from client proposal to endorsement, ordering, validation, and commit



Keywords

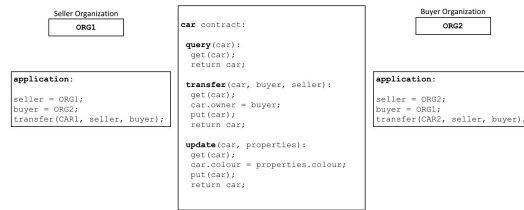
- **Fabric Smart Contracts** – Define common data models, business rules, and processes; they are packaged as **chaincodes**
- **World State** – The key-value store (database) that holds the current state of the ledger



Fabric Smart Contracts (Chaincode)

- Smart contracts are written in **Go**, **Node.js**, or **Java** and run in a separate Docker container from the peer
- Fabric supplies the **fabric-contract-api** (high-level API) for implementing chaincodes

Example: Car-Ownership Contract



The diagram illustrates two organizations (ORG1 and ORG2) each with an application that invokes a car-contract containing three functions: query, transfer, and update. The transfer function changes vehicle ownership across the two orgs.

Transaction Flow Reference: [Fabric Transaction Flow \(v2.2\)](#)

Writing Fabric Chaincode (Go Example)

Key points:

- **Supported languages:** Go, Node.js, Java
- **Execution model:** each chaincode runs in its own container, isolated from peers
- **API packages:** [github.com/hyperledger/fabric-contract-api-go/contractapi](https://github.com/hyperledger/fabric-contract-api-go) (Go), corresponding Node.js and Java SDKs

Code Walk-through:

```

// Define SmartContract
package main

import (
    "fmt"
    "github.com/hyperledger/fabric-contract-api-go/contractapi"
)

// SmartContract provides functions for storing and retrieving keys/values.
type SmartContract struct {
    contractapi.Contract
}

// InitLedger (optional in recent Fabric versions)
func (s *SmartContract) InitLedger(ctx contractapi.TransactionContextInterface) error {
    err := ctx.GetStub().PutState("testkey", []byte("testval"))
    if err != nil {
        return fmt.Errorf("Failed to put to world state: %s", err.Error())
    }
    return nil
}

// CreateKey – stores a key/value pair
func (s *SmartContract) CreateKey(ctx contractapi.TransactionContextInterface, key string, val string) error {
    return ctx.GetStub().PutState(key, []byte(val))
}

// QueryKey – reads a value by key
func (s *SmartContract) QueryKey(ctx contractapi.TransactionContextInterface, key string) (string, error) {
    val, err := ctx.GetStub().GetState(key)
    if err != nil {
        return "", fmt.Errorf("Failed to get from world state: %s", err.Error())
    }
    return string(val), nil
}

// Main entry point – start the chaincode
func main() {
    chaincode, err := contractapi.NewChaincode(new(SmartContract))
    if err != nil {
        fmt.Printf("Error creating chaincode: %s", err.Error())
        return
    }
}
  
```

```

if err = chaincode.Start(); err != nil {
    fmt.Printf("Error starting chaincode: %s", err.Error())
}
}

```

Key functions:

- **InitLedger** – optional bootstrap function
- **PutState / GetState** – low-level world-state key-value operations

Reference links:

- Go API: <https://pkg.go.dev/github.com/hyperledger/fabric-contract-api-go/contractapi>
- Node.js API: <https://hyperledger.github.io/fabric-chaincode-node/release-2.2/api/>
- Java API: <https://hyperledger.github.io/fabric-chaincode-java/release-2.2/api/org/hyperledger/fabric/contract/package-summary.html>

Conclusions (Lecture 36)

Point	Implication
Fabric chaincode can be authored in mainstream languages	Leverages familiar development tools
The ledger's world state is a simple key-value store	Any business logic can be encoded
Understanding the transaction flow is essential	Required before writing production-grade chaincode

Lecture 37 – Hyperledger Fabric 4

Instructor: Bishakh Chandra Ghosh

Scope

Topic	Focus
Fabric Application architecture	How client applications interact with the network
Fabric Certificate Authorities (CAs)	Identity management
Developing a DApp (Decentralised Application) on Fabric	Practical implementation

Fabric Application Stack

Client Application → Fabric SDK → Peer (Chaincode) → Ledger (World State)

Key components:

- **Peers** (endorsement, committing)
- **Ledger** (blockchain + world state)
- **Chaincode** (smart contracts)
- **Channels** (private sub-networks)
- **MSP (Membership Service Provider)** – defines organization identities
- **Fabric CA** – issues X.509 certificates for users, admins, and peers
- **Orderer** – consensus service

Development Prerequisites

SDK	Language
Node.js SDK	JavaScript/TypeScript
Java SDK	Java
Go SDK	Go
Python SDK	Python

Official documentation: https://hyperledger-fabric.readthedocs.io/en/release-2.2/getting_started.html#hyperledger-fabric-application-sdks

Sample Node.js DApp – Step-by-Step

4.1 Imports

```
const FabricCAServices = require('fabric-ca-client');
const { Wallets, Gateway } = require('fabric-network');
const fs = require('fs');
const path = require('path');
```

4.2 Connection Profile & CA

```
// Org1 connection profile
const ccpPath = path.resolve('../organizations/peerOrganizations/org1.example.com/connection-org1.json');
const ccp = JSON.parse(fs.readFileSync(ccpPath, 'utf8'));

// Org1 CA
const caInfo = ccp.certificateAuthorities['ca.org1.example.com'];
const caTLSCACerts = caInfo.tlsCACerts.pem;
const ca = new FabricCAServices(caInfo.url,
  { trustedRoots: caTLSCACerts, verify: false },
  caInfo.caName);
```

4.3 Enroll the CA Admin

```
const enrollment = await ca.enroll({ enrollmentID: 'admin', enrollmentSecret: 'adminpw' });
const x509Identity = {
  credentials: {
    certificate: enrollment.certificate,
    privateKey: enrollment.key.toBytes(),
  },
  mspId: 'Org1MSP',
  type: 'X.509',
};
await wallet.put('admin', x509Identity);
console.log('Admin enrolled and saved into wallet successfully');
```

4.4 Register & Enroll an Application User

```
const provider = wallet.getProviderRegistry().getProvider(adminIdentity.type);
const adminUser = await provider.getUserContext(adminIdentity, 'admin');

// Register the user
const secret = await ca.register({
  affiliation: 'org1.department1',
  enrollmentID: 'appUser',
  role: 'client'
}, adminUser);

// Enroll the user
const enrollment = await ca.enroll({ enrollmentID: 'appUser', enrollmentSecret: secret });
const x509Identity = {
  credentials: {
    certificate: enrollment.certificate,
    privateKey: enrollment.key.toBytes(),
  },
  mspId: 'Org1MSP',
  type: 'X.509',
};
```

```
await wallet.put('appUser', x509Identity);
console.log('Enrolled appUser and saved to wallet');
```

4.5 Connect to a Channel & Invoke Chaincode

```
const gateway = new Gateway();
await gateway.connect(ccp, { wallet, identity: 'appUser', discovery: { enabled: true, asLocalhost: true } });

const network = await gateway.getNetwork('mychannel');
const contract = network.getContract('keyvaluechaincode');

// Query
let result = await contract.evaluateTransaction('QueryKey', 'nptel');
console.log('First query:', result.toString());

// Invoke (create a new key)
await contract.submitTransaction('CreateKey', 'nptel', 'a new value');

// Verify update
result = await contract.evaluateTransaction('QueryKey', 'nptel');
console.log('Second query:', result.toString());

await gateway.disconnect();
```

Conclusions (Lecture 37)

Point	Explanation
Fabric Client SDK	Abstracts the complexities of network communication, endorsement, and ordering
Fabric CA	Issues cryptographic identities (X.509 certificates) that are stored in a wallet for later use
DApp operations	Can query and invoke chaincode functions using <code>evaluateTransaction</code> (read-only) and <code>submitTransaction</code> (state-changing) methods

Section 3: Byzantine Agreement Protocols and PBFT

This section covers *Practical Byzantine Fault Tolerance (PBFT)*, the classical *Byzantine Generals Problem*, and *consensus fundamentals* from an 80-page presentation ([source](#))



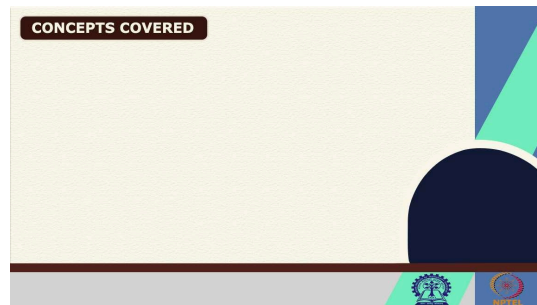
Lecture 31 – Byzantine Agreement Protocols

Course & Instructor: *Blockchain and its Applications*, Prof. Sandip Chakraborty, IIT Kharagpur

Practical Byzantine Fault Tolerance (PBFT)

PBFT Algorithm – A state-machine-replication protocol that tolerates Byzantine faults.

Quorum in PBFT – The minimum number of matching votes needed to move a request forward ($2f + 1$, where f is the maximum number of faulty replicas).



BFT Consensus – Classical Model

Lamport-Shostak-Pease (LSP) Algorithm – The seminal solution to the Byzantine Generals Problem (1982).

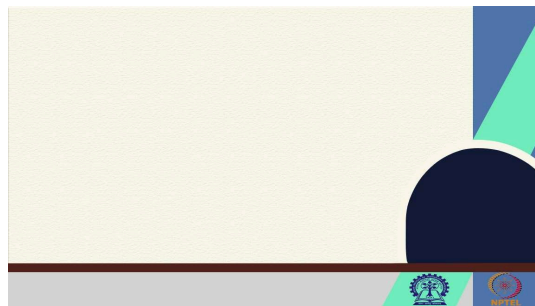
Assumptions of the classical model:

- Synchronous environment (known message-delivery bounds)
- Reliable communication channels
- Fully connected network (every replica can directly talk to every other)
- Receivers always know the identity of the sender

Citation: L Lamport, L Shostak, M Pease, "The Byzantine Generals Problem," *ACM Transactions on Programming Languages and Systems* 4.3 (1982): 382-401.

Why the Classical Assumptions Are Unrealistic:

- Real networks are **asynchronous** (no guaranteed delivery time)
- Links can be unreliable; nodes may crash or act maliciously



From Classical to Practical – PBFT

Classical BFT	Practical BFT (PBFT)
Synchronous, reliable channels	Asynchronous environment – safety prioritized over liveness
No cryptography	Digital signatures to authenticate senders
High overhead	Low overhead through efficient message patterns
Not used in production	Adopted by many blockchain platforms (e.g., Tendermint, Hyperledger Fabric)

Key Publications: M Castro & B Liskov, "Practical Byzantine Fault Tolerance," *USENIX OSDI* 1999.

PBFT Overview

State-Machine Replication: All correct replicas maintain an identical state machine.

Parameter	Value/Description
Replica Count	$3f + 1$ replicas guarantee tolerance of up to f Byzantine faults
Views	A <i>view</i> is a configuration identified by an integer v
Primary	One replica is the leader (primary) in each view
Backups	The remaining replicas are backups

View Change: When the primary is suspected faulty, replicas shift to a new view with a new primary.

PBFT Message Flow (Three-Phase Commit)

Phase	Description
1. Request	Client → Primary
2. Pre-Prepare	Primary assigns a sequence number n and multicasts β_p to all backups (where d = digest of the request, β_p = primary's private key)
3. Prepare	Backups broadcast β_k to all replicas (including primary) as proof of agreement on the ordering
4. Commit	After receiving $2f + 1$ matching Prepare messages , each replica sends β_k

Commit Condition: A replica commits when it has sent its own Commit and has received **$2f + 1$ Commit messages** from others.

Client Decision: The client waits for **$2f + 1$ replies** (from distinct replicas) before considering the operation successful.

Quorum Reasoning (Why $2f + 1$?)

Byzantine Dissemination Quorum: Any two quorums intersect in at least one correct replica.

Property	Explanation
Availability	At least one quorum of non-faulty replicas always exists
Safety	Guarantees that two conflicting decisions cannot both gather a quorum

Fault Scenarios:

Fault Type	Message Reception	Quorum Needed	Reasoning
Crash / Network loss ($\leq f$)	Missing messages from faulty replicas	$2f + 1$ Prepare from <i>non-faulty</i> replicas	All votes are correct; consensus reached
Byzantine ($\leq f$)	Faulty replicas may send arbitrary messages	$2f + 1$ Prepare (may include up to f bad votes)	Still guarantees at least $f + 1$ correct votes, enough to outvote the faulty ones

Note: Because the network is asynchronous, messages can be delayed or arrive out of order; the protocol must buffer and track messages until the quorum is satisfied.

Lecture 32 – Safety and Liveness of PBFT

Key Topics

Topic	Focus
-------	-------

Safety	No two correct replicas decide different values
Liveness	The system eventually makes progress (decides)
PBFT View-Change	Mechanism to replace a faulty primary
Weak Synchrony Assumption	Minimal timing guarantees needed for liveness

Safety in PBFT

Single Primary: Unlike Multi-Paxos, PBFT uses one leader per view, avoiding "ping-pong" conflicts.

Quorum of $2f + 1$: Guarantees that the primary (or any replica) always has a majority of votes supporting its proposal, even if some replicas are silent due to asynchrony.

Liveness in PBFT

Issue	Explanation
Primary Failure	If the primary crashes or becomes Byzantine, progress halts because replicas wait for its messages
Pure Asynchrony Limitation	In a fully asynchronous model, you cannot distinguish a slow primary from a failed one, so liveness cannot be guaranteed

Weak Synchrony Assumption

Condition	Description
1. Correct Sender & Receiver	Both endpoints behave according to the protocol
2. Retransmission	The sender repeatedly transmits until the receiver acknowledges receipt
3. Bounded Delay	There exists an (unknown) upper bound on message transmission time

These modest timing guarantees are sufficient to detect a stalled primary and trigger a view change.

View-Change Protocol

Step	Action
1. Timeout Detection	Each backup starts a timer when it receives a request; the timer resets on new requests or when the request is executed
2. Trigger	If the timer expires while the replica is still in view v_i , it suspects the primary and initiates a VIEW-CHANGE
3. VIEW-CHANGE Message	β_k where:
	• n = last stable checkpoint sequence number known to replica k
	• C = set of $2f + 1$ valid checkpoint messages for n
	• P = set of prepared requests with sequence numbers $> n$ (each request m has a prepared set P_m)
4. Quorum for New View	When a replica receives $2f + 1$ VIEW-CHANGE messages, it can form a new view
5. Primary Selection	Common policies: Round-Robin (e.g., Hyperledger Sawtooth) or Leader Election (e.g., Hyperledger Fabric)
6. VIEW-CHANGE-ACK & NEW-VIEW	Replicas acknowledge the new view; once a quorum of ACKs is collected, the new primary broadcasts a NEW-VIEW message to start processing new requests

Section 4: Enterprise Blockchains and Consensus Scalability

This section examines enterprise blockchain characteristics, consensus scalability challenges, and the trade-offs between PoW and PBFT from a 96-page presentation ([source](#))

Lecture 33 – Enterprise Blockchains

Instructor: Prof. Sandip Chakraborty

Scope & Motivation

Enterprise blockchains aim to bring the core benefits of distributed ledger technology (DLT) to business contexts.

Application Space: Beyond Bitcoin, blockchains can redesign fundamental business interactions, enabling new digital transaction models.

Business-Network Asset Transfer

Traditional Approach	Blockchain Solution
Inefficient, expensive, and vulnerable (fragmented records across parties A-D, banks, auditors)	Consensus, provenance, immutability, and finality solve the above challenges

Core Benefits

Benefit	What it Means for Business
Shared Ledger	Append-only, distributed system of record shared across the network
Smart Contracts	Business logic embedded in the transaction layer; automatic enforcement
Security	Transactions are authenticated, tamper-proof, and verifiable
Consensus	All participants agree on the state of the ledger
Operational Gains	Faster transaction times (days → near-instant), reduced intermediaries, lower fraud risk, and new business models
IoT Integration	Seamless supply-chain tracking and automation

Degree of Centralization

- A figure (source: UK Government Chief Scientific Adviser report) illustrates a spectrum from **fully centralized** to **fully decentralized** systems, with enterprise blockchains positioned in the middle (permissioned, semi-centralized)

Permissionless vs. Permissioned Blockchains

Aspect	Permissionless	Permissioned
Access	Open to anyone	Restricted to known participants
Identity	Anonymous or pseudonymous	Known, authenticated identities
Consensus	PoW, PoS, etc.	PBFT, Raft, etc.
Performance	Lower throughput, higher latency	Higher throughput, lower latency
Use case	Cryptocurrency, public DApps	Enterprise, consortium applications

Lecture 38 – Consensus Scalability

Instructor: Prof. Sandip Chakraborty

Overview

- Examination of **scalability challenges** in blockchain consensus mechanisms
- Comparison of **Proof-of-Work (PoW)** vs. **Practical Byzantine Fault Tolerance (PBFT)**
- Discussion of **finality** and throughput limits

Consensus Protocol Taxonomy

Category	Protocols
Permissionless	PoW, Proof-of-Stake (PoS), Proof-of-Burn (PoB), Proof-of-Elapsed-Time (PoET)
Permissioned	Byzantine Agreement (e.g., PBFT)

PoW vs. PBFT

Feature	PoW	PBFT
Environment	Open, large-scale, permissionless	Closed, permissioned consortium
Scalability (nodes)	Scales to many nodes (e.g., Bitcoin)	Not scalable beyond a few dozen nodes
Throughput	Low (few TPS)	High (hundreds to thousands TPS)
Finality	Probabilistic – forks may occur	Deterministic – total ordering guarantees finality

PoW Scalability – Bitcoin Example

Parameter	Value
Block frequency	10 minutes (600 seconds)
Block size	1 MiB ($\approx 1,048,576$ bytes)
Average transaction size	380.04 bytes

Calculations:

Transactions per block = $1,048,576 / 380.04 \approx 2,759.12$
 TPS = $2,759.12 / 600 \approx 4.6$

Visa benchmark: Visa processes ~1,700 TPS on average with peaks of 4,000 TPS – Bitcoin's ~4.6 TPS is far below commercial requirements.

Section 5: Permissioned Blockchains, Smart Contracts, and Paxos Consensus

This section covers permissioned blockchain models, smart contract execution, state-machine replication, and the Paxos consensus algorithm from a 177-page presentation ([source](#))



Permissioned Blockchains

Definition

Permissioned Model: A blockchain architecture where every participant is **authenticated a priori**.

Characteristic	Description
Membership management	Controlled by a Membership Service Provider (MSP) that issues credentials and controls who can join the network
Identity	Participants know each other's identities , but trust is not assumed
Security	Security and consensus mechanisms are still required despite known participants

Why Use Permissioned Blockchains?

Ideal for **business applications** where contracts are executed among a **closed set of known participants** (e.g., supply-chain provenance tracking).

Smart Contracts over Closed Networks

What Is a Smart Contract?

"A self-executing contract in which the terms of the agreement between the buyer and the seller are directly written into the lines of code." – *scalablockchain.com*

Execution Flow in a Permissioned Setting

Step	Description
1. Store	The contract code is stored on the blockchain
2. Trigger	An external event (e.g., receipt of goods) triggers execution
3. Local Execution	Each peer runs the contract locally
4. Transaction Generation	The execution produces one or more transactions as output
5. Validation & Commit	Peers validate the generated transactions; once approved, they are committed to the ledger, possibly triggering further events

Do All Peers Need to Execute?

Observation	Proposed Approach
Running the contract on every peer is costly	Execute on a subset of nodes and propagate the resulting state to the rest

Requirements for Subset Execution:

1. **Majority agreement** on the resulting state
2. **Proof generation** that a peer has agreed on the state of execution (e.g., cryptographic receipts)

State-Machine Replication (SMR)

Core Idea

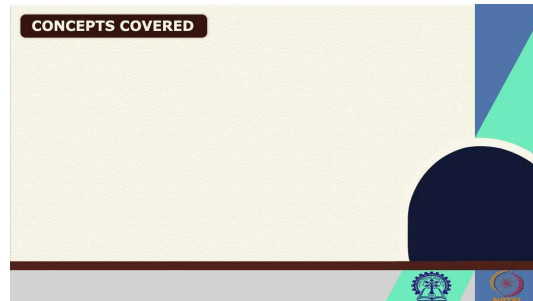
Any **deterministic program** (including a smart contract) can be modeled as a **state machine**. **SMR** replicates this state machine across multiple independent servers (or peers).

Example State-Machine (Pseudo-code)

```
S1: while (moreGoods == 1) DeliverGoods();
S2: if (allOrderComplete == 0) goto S1;
    else { printf("Goods transfer complete"); }
```

Replication Workflow

1. **Transaction ordering** – all replicas receive the same ordered list of transactions (T1, T2, ...)
2. **Independent execution** – each replica executes the ordered transactions on its local copy of the state machine
3. **Consensus on final state** – replicas must **agree** on the final state after execution (e.g., state **S2**)



💛 Consensus in Permissioned Blockchains

Why Consensus Is Still Needed

Even when participants are known, **faults** (crash or malicious) can cause divergent states. Consensus ensures **agreement on transaction ordering and final state** across all replicas.

Types of Faults

Fault Type	Description
Crash Fault	Node stops operating (hardware/software failure). In asynchronous settings, it is indistinguishable from a delayed message
Byzantine Fault	Node behaves arbitrarily, sending conflicting information to different peers (e.g., malicious or buggy)

Consensus Goals (Asynchronous CFT)

1. **Validity**: If all correct processes propose the same value v , any correct process decides v
2. **Agreement**: No two correct processes decide differently (safety)
3. **Termination**: Every correct process eventually decides (liveness)

Note: The FLP impossibility theorem tells us that in a purely asynchronous system, we cannot guarantee both safety and liveness simultaneously; practical algorithms prioritize safety.

🇮🇳 Classical Crash-Fault-Tolerant (CFT) Consensus

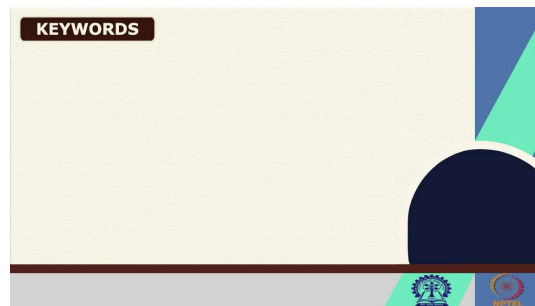
Core Algorithms

Algorithm	Key Traits	Typical Use
Paxos (Lamport, 1989)	Most fundamental CFT; works with $2F + 1$ nodes to tolerate F crash faults	DynamoDB, many distributed key-value stores
Raft	Simpler presentation of the same ideas as Paxos; easier to implement	Hyperledger Fabric transaction ordering service

Node Roles in Paxos

Role	Function
Proposer	Initiates a proposal (value)
Acceptor	Votes on proposals; maintains the highest promised proposal ID
Learner	Learns the decided value after a majority of acceptors have accepted

A single physical node can play any combination of these roles.



Majority Requirements

- With **F crash faults**, **2F + 1** total nodes guarantee a majority of **F + 1** correct acceptors
- Two majorities always **overlap** in at least one node, ensuring that any two conflicting proposals cannot both be accepted

Paxos Algorithm – Step-by-Step

Phase 1 – Prepare

1. **Proposer** selects a unique, monotonically increasing **proposal ID** (IDp)
 - Example generation: hash(timestamp || proposerID)
2. Sends **PREPARE(IDp)** to a **majority** (or all) of the acceptors

Phase 1 – Promise

- **Acceptor** receives **PREPARE(IDp)**:
 - If it has already promised a higher ID, it **ignores** the message
 - Otherwise, it **promises** not to accept any proposal with an ID lower than IDp and replies with **PROMISE(IDp)** (including any previously accepted value and its ID, if any)

Phase 2 – Accept-Request

- After receiving a **majority of PROMISEs**, the proposer sends **ACCEPT-REQUEST(IDp, VALUE)** to the same (or all) acceptors
- The **VALUE** may be:
 - The **highest-numbered previously accepted value** (if any), or
 - Any value of the proposer's choice (if no prior accepted value)

Phase 2 – Accept

- **Acceptor** checks the promise:
 - If it has already promised a higher ID, it **ignores**
 - Otherwise, it records the accepted pair **(IDp, VALUE)** and sends **ACCEPT(IDp, VALUE)** to all **learners**

Decision

- Once a **majority of acceptors** send **ACCEPT** for the same **(IDp, VALUE)**, **learners** deem the **VALUE** decided
- **Consensus is on the VALUE**, not on the ID

Example Flow (Dinner vs. Movie):

```
Proposer → PREPARE 100 → Majority → PROMISE 100
Proposer → ACCEPT-REQUEST 100, "Dinner" → Majority → ACCEPT 100, "Dinner"
Learners learn "Dinner" as the decided value.
```


Handling Multiple Proposers

- **Concurrent proposals** may have different IDs (e.g., 100 and 150)
- An acceptor that has already **accepted** a lower-ID proposal must include that accepted pair in its **PROMISE** response to a higher-ID **PREPARE**
- The higher-ID proposer must **adopt the highest previously accepted value** when sending its **ACCEPT-REQUEST**

Pseudocode for Proposer Decision:

```
if (majority PROMISEs contain accepted value with highest IDa) {  
    VALUE = VALUEa // adopt highest accepted value  
} else {  
    VALUE = valueChosenByProposer  
}  
send ACCEPT-REQUEST(IDp, VALUE)
```

Majority Voting Illustrated

The slides use a **dinner/movie negotiation** scenario to demonstrate how different message interleavings affect which proposal gains a majority.

Case	Outcome
Case 1: Majority for ID 100	ID 50 cannot succeed because the same acceptors have already promised to ignore lower IDs
Case 2: If a majority eventually promises for ID 50	ID 50 can become the decided value ("Movie") because some acceptors ignored the higher ID

These illustrations emphasize the **overlap property of majorities** and how **higher IDs dominate lower ones** unless the higher proposal fails to achieve a majority.

Liveness Considerations

- **Paxos guarantees safety** (no two different values are decided)
- **Liveness** (eventual decision) is **not guaranteed** in purely asynchronous environments because a proposer may keep being pre-empted by higher-ID proposals
- Practical systems often introduce **leader election** or **timeouts** to improve liveness

Multi-Paxos (Continuous Consensus)

- Real-world applications require a **stream of agreed values** (e.g., transaction logs)
- **Multi-Paxos** runs a **sequence of Paxos instances**, each identified by a **round number** (or instance ID)
- Once a value is decided for **Round n**, acceptors **ignore** any conflicting accept request for the same round

Leader-Based Optimization

A **stable leader** (single proposer) can **skip Phase 1** for subsequent rounds after the initial round, dramatically reducing latency.

Byzantine Fault Tolerance (BFT)

Byzantine Generals Problem

Scenario: A commander and several lieutenants must agree on an attack/retreat order despite possible traitors.

Configuration	Outcome
---------------	---------

One faulty lieutenant among two lieutenants	Consensus is impossible (illustrated through multiple diagrams)
General case	F Byzantine faults require at least $3F + 1$ total nodes to achieve consensus

Byzantine Agreement Protocols

BFT Consensus algorithms (e.g., PBFT, Tendermint) extend the SMR approach with additional message phases and cryptographic signatures to tolerate malicious behavior.

Comparison with Paxos

Protocol	Fault Model	Node Requirement
Paxos	Crash faults only	$2F + 1$ nodes for F faults
Byzantine-tolerant algorithms	Byzantine (arbitrary) faults	$3F + 1$ nodes for F faults

Why more nodes? Faulty nodes can send contradictory messages, creating network partitions that require additional redundancy to overcome.

🔒 Security Considerations for Paxos

- **Malicious Acceptors** may send conflicting **PROMISE** or **ACCEPT** messages
- Because **majorities overlap**, a single malicious node cannot force an incorrect decision on its own
- However, if $\geq F$ nodes are malicious (where $F \geq (N-1)/3$), the safety guarantees of Paxos break down, prompting the need for BFT protocols

📊 Summary of Key Take-aways

Topic	Core Insight
Permissioned Model	Authenticated participants still need consensus; membership is managed by an MSP
Smart Contracts in Closed Networks	Execution can be limited to a subset of nodes with cryptographic proof of state agreement
State-Machine Replication	Any deterministic contract can be modeled as a state machine; SMR provides a natural foundation for consensus
Consensus Requirements	Crash-fault tolerant consensus needs $2F + 1$ nodes; Byzantine-fault tolerant needs $3F + 1$ nodes

Section 5: Permissioned Blockchain Consensus and Paxos Algorithm

This section covers permissioned blockchain fundamentals, smart contract execution models, state-machine replication, and the complete Paxos consensus protocol from a 177-page presentation ([source](#))



Permissioned Blockchains

Definition

Permissioned Model: A blockchain architecture where every participant is **authenticated a priori**.

Characteristic	Description
Membership management	Controlled by a Membership Service Provider (MSP) that issues credentials and controls who can join the network
Identity	Participants know each other's identities , but trust is not assumed
Security requirement	Security and consensus mechanisms are still required

Why Use Permissioned Blockchains?

Ideal for **business applications** where contracts are executed among a **closed set of known participants** (e.g., supply-chain provenance tracking).

Smart Contracts over Closed Networks

What Is a Smart Contract?

"A self-executing contract in which the terms of the agreement between the buyer and the seller are directly written into the lines of code." – *scalablockchain.com*

Execution Flow in a Permissioned Setting

Step	Description
1. Store	The contract code is stored on the blockchain
2. Trigger	An external event (e.g., receipt of goods) triggers execution
3. Local Execution	Each peer runs the contract locally
4. Transaction Generation	The execution produces one or more transactions as output
5. Validation & Commit	Peers validate the generated transactions; once approved, they are committed to the ledger, possibly triggering further events

Do All Peers Need to Execute?

Observation	Proposed Approach
Running the contract on every peer is costly	Execute on a subset of nodes and propagate the resulting state to the rest

Requirements for Subset Execution:

1. **Majority agreement** on the resulting state
2. **Proof generation** that a peer has agreed on the state of execution (e.g., cryptographic receipts)

State-Machine Replication (SMR)

Core Idea

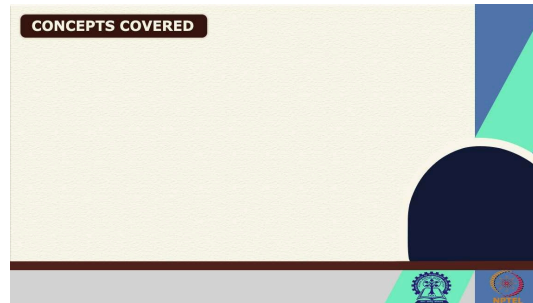
Any **deterministic program** (including a smart contract) can be modeled as a **state machine**. **SMR** replicates this state machine across multiple independent servers (or peers).

Example State-Machine (Pseudo-code)

```
S1: while (moreGoods == 1) DeliverGoods();
S2: if (allOrderComplete == 0) goto S1;
    else { printf("Goods transfer complete"); }
```

Replication Workflow

1. **Transaction ordering** – all replicas receive the same ordered list of transactions (T1, T2, ...)
2. **Independent execution** – each replica executes the ordered transactions on its local copy of the state machine
3. **Consensus on final state** – replicas must **agree** on the final state after execution (e.g., state **S2**)



💛 Consensus in Permissioned Blockchains

Why Consensus Is Still Needed

Even when participants are known, **faults** (crash or malicious) can cause divergent states. Consensus ensures **agreement on transaction ordering and final state** across all replicas.

Types of Faults

Fault Type	Description
Crash Fault	Node stops operating (hardware/software failure). In asynchronous settings, it is indistinguishable from a delayed message
Byzantine Fault	Node behaves arbitrarily, sending conflicting information to different peers (e.g., malicious or buggy)

Consensus Goals (Asynchronous CFT)

1. **Validity**: If all correct processes propose the same value v , any correct process decides v
2. **Agreement**: No two correct processes decide differently (safety)
3. **Termination**: Every correct process eventually decides (liveness)

Note: The FLP impossibility theorem tells us that in a purely asynchronous system, we cannot guarantee both safety and liveness simultaneously; practical algorithms prioritize safety.

🇮🇳 Classical Crash-Fault-Tolerant (CFT) Consensus

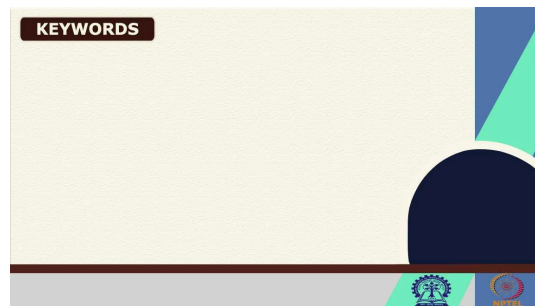
Core Algorithms

Algorithm	Key Traits	Typical Use
Paxos (Lamport, 1989)	Most fundamental CFT; works with $2F + 1$ nodes to tolerate F crash faults	DynamoDB, many distributed key-value stores
Raft	Simpler presentation of the same ideas as Paxos; easier to implement	Hyperledger Fabric transaction ordering service

Node Roles in Paxos

Role	Function
Proposer	Initiates a proposal (value)
Acceptor	Votes on proposals; maintains the highest promised proposal ID
Learner	Learns the decided value after a majority of acceptors have accepted

A single physical node can play any combination of these roles.



Majority Requirements

- With **F crash faults**, **2F + 1** total nodes guarantee a majority of **F + 1** correct acceptors
- Two majorities always **overlap** in at least one node, ensuring that any two conflicting proposals cannot both be accepted

Paxos Algorithm – Step-by-Step

Phase 1 – Prepare

- Proposer** selects a unique, monotonically increasing **proposal ID** (IDp)
 - Example generation: `hash(timestamp || proposerID)`
- Sends **PREPARE(IDp)** to a **majority** (or all) of the acceptors

Phase 1 – Promise

- Acceptor** receives **PREPARE(IDp)**:
 - If it has already promised a higher ID, it **ignores** the message
 - Otherwise, it **promises** not to accept any proposal with an ID lower than IDp and replies with **PROMISE(IDp)** (including any previously accepted value and its ID, if any)

Phase 2 – Accept-Request

- After receiving a **majority of PROMISEs**, the proposer sends **ACCEPT-REQUEST(IDp, VALUE)** to the same (or all) acceptors
- The **VALUE** may be:
 - The **highest-numbered previously accepted value** (if any), or
 - Any value of the proposer's choice (if no prior accepted value)

Phase 2 – Accept

- Acceptor** checks the promise:
 - If it has already promised a higher ID, it **ignores**
 - Otherwise, it records the accepted pair **(IDp, VALUE)** and sends **ACCEPT(IDp, VALUE)** to all **learners**

Decision

- Once a **majority of acceptors** send **ACCEPT** for the same **(IDp, VALUE)**, **learners** deem the **VALUE** decided
- Consensus is on the VALUE**, not on the ID

Example Flow (Dinner vs. Movie):

```
Proposer → PREPARE 100 → Majority → PROMISE 100
Proposer → ACCEPT-REQUEST 100, "Dinner" → Majority → ACCEPT 100, "Dinner"
Learners learn "Dinner" as the decided value.
```

Handling Multiple Proposers

- **Concurrent proposals** may have different IDs (e.g., 100 and 150)
- An acceptor that has already **accepted** a lower-ID proposal must include that accepted pair in its **PROMISE** response to a higher-ID **PREPARE**
- The higher-ID proposer must **adopt the highest previously accepted value** when sending its **ACCEPT-REQUEST**

Pseudocode for Proposer Decision:

```
if (majority PROMISEs contain accepted value with highest IDa) {  
    VALUE = VALUEa // adopt highest accepted value  
} else {  
    VALUE = valueChosenByProposer  
}  
send ACCEPT-REQUEST(IDp, VALUE)
```

Majority Voting Illustrated

The slides use a **dinner/movie negotiation** scenario to demonstrate how different message interleavings affect which proposal gains a majority.

Case	Outcome
Case 1: Majority for ID 100	ID 50 cannot succeed because the same acceptors have already promised to ignore lower IDs
Case 2: If a majority eventually promises for ID 50	ID 50 can become the decided value ("Movie") because some acceptors ignored the higher ID

These illustrations emphasize the **overlap property of majorities** and how **higher IDs dominate lower ones** unless the higher proposal fails to achieve a majority.

Liveness Considerations

- **Paxos guarantees safety** (no two different values are decided)
- **Liveness** (eventual decision) is **not guaranteed** in purely asynchronous environments because a proposer may keep being pre-empted by higher-ID proposals
- Practical systems often introduce **leader election** or **timeouts** to improve liveness

Multi-Paxos (Continuous Consensus)

- Real-world applications require a **stream of agreed values** (e.g., transaction logs)
- **Multi-Paxos** runs a **sequence of Paxos instances**, each identified by a **round number** (or instance ID)
- Once a value is decided for **Round n**, acceptors **ignore** any conflicting accept request for the same round

Leader-Based Optimization

A **stable leader** (single proposer) can **skip Phase 1** for subsequent rounds after the initial round, dramatically reducing latency.

Byzantine Fault Tolerance (BFT)

Byzantine Generals Problem

Scenario: A commander and several lieutenants must agree on an attack/retreat order despite possible traitors.

Configuration	Outcome
---------------	---------

One faulty lieutenant among two lieutenants	Consensus is impossible (illustrated through multiple diagrams)
General case	F Byzantine faults require at least $3F + 1$ total nodes to achieve consensus

Byzantine Agreement Protocols

BFT Consensus algorithms (e.g., PBFT, Tendermint) extend the SMR approach with additional message phases and cryptographic signatures to tolerate malicious behavior.

Comparison with Paxos

Protocol	Fault Model	Node Requirement
Paxos	Crash faults only	$2F + 1$ nodes for F faults
Byzantine-tolerant algorithms	Byzantine (arbitrary) faults	$3F + 1$ nodes for F faults

Why more nodes? Faulty nodes can send contradictory messages, creating network partitions that require additional redundancy to overcome.

🔒 Security Considerations for Paxos

- **Malicious Acceptors** may send conflicting **PROMISE** or **ACCEPT** messages
- Because **majorities overlap**, a single malicious node cannot force an incorrect decision on its own
- However, if $\geq F$ nodes are malicious (where $F \geq (N-1)/3$), the safety guarantees of Paxos break down, prompting the need for BFT protocols

📊 Summary of Key Take-aways

Topic	Core Insight
Permissioned Model	Authenticated participants still need consensus; membership is managed by an MSP
Smart Contracts in Closed Networks	Execution can be limited to a subset of nodes with cryptographic proof of state agreement
State-Machine Replication	Any deterministic contract can be modeled as a state machine; SMR provides a natural foundation for consensus
Consensus Requirements	Crash-fault tolerant consensus needs $2F + 1$ nodes; Byzantine-fault tolerant needs $3F + 1$ nodes

🎯 Putting it All Together

🔗 Interoperability & Asset Transfer

- **Atomic swap** protocols use a combined **hash-lock + time-lock** (HTLC) to guarantee that a cross-chain exchange is *all-or-nothing*.

An atomic transaction is indivisible: either every leg completes or none does.

- The naïve **trusted-third-party (TTP)** model (centralised exchanges) offers speed but introduces single points of failure, as illustrated by historic hacks (Mt. Gox, Bitfinex).
- **Multi-party cross-chain swaps** extend the HTLC pattern to more than two ledgers, preserving decentralisation while scaling asset movement.

🏰 Consensus Foundations

- **ByzCoin** merges **Proof-of-Work (PoW)** membership proofs with **PBFT-style voting**, using **CoSi** (tree-based multicast + Schnorr multi-signatures) to reduce communication from $O(n^2)$ to $O(\log n)$.
- **Algorand** achieves **pure proof-of-stake (PPoS)** via **cryptographic sortition** (VRF-derived selection) and the **BA*** two-phase agreement, delivering near-instant finality under weak synchrony.
- Classical **Paxos/Raft** (crash-fault tolerant) rely on $2f + 1$ quorum for f crash faults; **PBFT** (Byzantine-fault tolerant) needs $3f + 1$ nodes and a $2f + 1$ quorum to preserve safety.
- **View-change** mechanisms (PBFT) and **leader election** (Algorand, Multi-Paxos) provide liveness when the primary becomes unresponsive.

Decentralized Identity

- **Self-Sovereign Identity (SSI)** puts the holder in control of verifiable credentials, eliminating reliance on a single identity provider.
- **Decentralised Identifiers (DIDs)** are globally unique, cryptographically verifiable URIs stored on a **verifiable data registry** (often a blockchain).

A DID Document contains public keys, authentication methods, and service endpoints, enabling trust-less verification of the holder.

- **Hyperledger Indy** implements DIDs and verifiable credentials, supporting privacy-preserving proofs (zero-knowledge) for credential presentations.

Permissioned Enterprise Blockchains

- **Hyperledger Fabric** separates **ordering** from **execution**; chaincode (smart contracts) can be written in Go, Node.js, or Java and runs in isolated Docker containers.
- The **Fabric CA** issues X.509 certificates, populating the **Membership Service Provider (MSP)** so that every participant is authenticated before joining a channel.
- **State-Machine Replication (SMR)** underpins both Fabric and permissioned BFT protocols: deterministic contract code is executed locally on each replica after a common transaction order is agreed upon.
- To reduce overhead, execution can be limited to a **subset of endorsing peers** with cryptographic receipts proving agreement before the resulting state is committed network-wide.

Comparative Landscape

Dimension	Permissionless (Public)	Permissioned (Enterprise)
Membership	Open, PoW or PoS-based proof	Authenticated via MSP / CA
Typical Consensus	PoW, PoS, Algorand's BA*	PBFT/ByzCoin (BFT), Paxos/Raft (CFT)
Fault Model	Primarily Byzantine (malicious miners)	Can tolerate crash or Byzantine depending on protocol
Finality	Probabilistic (multiple confirmations)	Deterministic (instant after commit)
Scalability Mechanisms	Sharding, Layer-2, CoSi aggregation	Tree-based multicast, leader-based Multi-Paxos, subset execution
Trust Assumptions	Trust-less, incentive-driven	Trust among known entities, but no implicit trust—cryptographic guarantees still required
Use Cases	Public cryptocurrencies, open DeFi	Supply-chain provenance, inter-company asset transfer, private data sharing